



PROYECTO FINAL

Efren Aaron Gomez Casimiro

CLASES Y FUNCIONES

Tarea

- Esta es la clase base para todas las tareas.
- Guarda el **nombre** y el **departamento** al que pertenece la tarea.
- Los métodos `equals()` y `hashCode()` se sobrescriben para asegurar que dos objetos **Tarea** se consideren iguales si tienen el mismo nombre y departamento, lo cual es útil para su uso en colecciones como **HashSet** o **HashMap**.

```
14
15 // --- CLASE 1: Tarea (Modificada) ---
16 class Tarea {
17     private String nombre;
18     private String departamento;
19
20     public Tarea(String nombre, String departamento) {
21         this.nombre = nombre;
22         this.departamento = departamento;
23     }
24
25     public String getNombre() {
26         return nombre;
27     }
28
29     public String getDepartamento() {
30         return departamento;
31     }
32
33     @Override
34     public String toString() {
35         return "Tarea: " + nombre + " (Departamento: " + departamento + ")";
36     }
37
38     @Override
39     public boolean equals(Object o) {
40         if (this == o) return true;
41         if (o == null || getClass() != o.getClass()) return false;
42         Tarea tarea = (Tarea) o;
43         return nombre.equals(tarea.nombre) && departamento.equals(tarea.departamento);
44     }
45
46     @Override
47     public int hashCode() {
48         return nombre.hashCode() + departamento.hashCode();
49     }
50 }
```

```
// --- CLASE NUEVA: Tarea con Prioridad ---
class TareaConPrioridad extends Tarea {
    private int prioridad; // 1 = Alta, 2 = Media, 3 = Baja
    private String fechaEntrega;

    public TareaConPrioridad(String nombre, String departamento, int prioridad, String fechaEntrega) {
        super(nombre, departamento);
        this.prioridad = prioridad;
        this.fechaEntrega = fechaEntrega;
    }

    public int getPrioridad() {
        return prioridad;
    }

    public String getFechaEntrega() {
        return fechaEntrega;
    }

    @Override
    public String toString() {
        return "Tarea: " + getNombre() + " (Depto: " + getDepartamento() + ", Prioridad: " + prioridad + ", Entrega: " + fechaEntrega;
    }
}
```

TareaConPrioridad

- Esta clase **hereda** de **Tarea**.
- Añade dos nuevos atributos: **prioridad** (un número del 1 al 3) y **fecha de entrega**.
- Sobrescribe el método **toString()** para incluir la nueva información en la descripción de la tarea.

Empleado

- Representa a un empleado con un **nombre** y un **departamento**.
- Contiene una lista de **tareas asignadas**, lo que permite llevar un registro de las responsabilidades de cada empleado.

```
// --- CLASE NUEVA: Empleado ---
class Empleado {
    private String nombre;
    private String departamento;
    private List<Tarea> tareasAsignadas;

    public Empleado(String nombre, String departamento) {
        this.nombre = nombre;
        this.departamento = departamento;
        this.tareasAsignadas = new ArrayList<>();
    }

    public String getNombre() {
        return nombre;
    }

    public String getDepartamento() {
        return departamento;
    }

    public List<Tarea> getTareasAsignadas() {
        return tareasAsignadas;
    }

    public void asignarTarea(Tarea tarea) {
        this.tareasAsignadas.add(tarea);
    }

    @Override
    public String toString() {
        return "Empleado: " + nombre + " (Depto: " + departamento + ")";
    }
}
```

GrafoTareas

```
// --- CLASE NUEVA: Grafo para Dependencias de Tareas ---
class GrafoTareas {
    private Map<Tarea, Set<Tarea>> adjList;

    public GrafoTareas() {
        this.adjList = new HashMap<>();
    }

    public void agregarTarea(Tarea tarea) {
        adjList.putIfAbsent(tarea, new HashSet<>());
    }

    public void agregarDependencia(Tarea tarea, Tarea tareaDependiente) {
        agregarTarea(tarea);
        agregarTarea(tareaDependiente);
        adjList.get(tarea).add(tareaDependiente);
        System.out.println("⚡ Dependencia agregada: " + tareaDependiente.getNombre() + " depende de " + tarea.getNombre());
    }

    public Set<Tarea> getTareasDependientes(Tarea tarea) {
        return adjList.getOrDefault(tarea, new HashSet<>());
    }

    public void mostrarGrafo() {
        System.out.println("--- GRAFO DE DEPENDENCIAS ---");
        for (Map.Entry<Tarea, Set<Tarea>> entry : adjList.entrySet()) {
            System.out.print(entry.getKey().getNombre() + " -> [");
            boolean first = true;
            for (Tarea dependiente : entry.getValue()) {
                if (!first) {
                    System.out.print(", ");
                }
                System.out.print(dependiente.getNombre());
                first = false;
            }
            System.out.println("]");
        }
    }
}
```

- Implementa un **grafo dirigido** para modelar las dependencias entre tareas.
- **adjList**: Es un **HashMap** donde cada tarea (**clave**) se asocia con un **HashSet** de las tareas que dependen de ella (**valor**).
- **agregarDependencia()**: Conecta dos tareas, indicando que una tarea dependiente no puede comenzar hasta que la tarea principal esté terminada.
- **mostrarGrafo()**: Imprime una representación de las dependencias.

ArbolBinarioEmpleados

- Implementa un **árbol binario de búsqueda** para organizar a los empleados.
- El ordenamiento se realiza por el **departamento** del empleado.
- **insertar()**: Agrega un nuevo empleado al árbol en la posición correcta.
- **buscarPorDepartamento()**: Permite encontrar de manera eficiente todos los empleados que pertenecen a un departamento específico.

```
// --- CLASE NUEVA: Nodo para Árbol Binario ---
class NodoEmpleado {
    Empleado empleado;
    NodoEmpleado izquierda;
    NodoEmpleado derecha;

    public NodoEmpleado(Empleado empleado) {
        this.empleado = empleado;
        this.izquierda = null;
        this.derecha = null;
    }
}

// --- CLASE NUEVA: Árbol Binario de Búsqueda ---
class ArbolBinarioEmpleados {
    private NodoEmpleado raiz;

    public ArbolBinarioEmpleados() {
        this.raiz = null;
    }

    // Método para insertar un empleado de forma recursiva
    public void insertar(Empleado empleado) {
        this.raiz = insertarRecursivo(this.raiz, empleado);
        System.out.println("✅ Empleado agregado (Árbol Binario): " + empleado.getNombre());
    }

    private NodoEmpleado insertarRecursivo(NodoEmpleado nodo, Empleado empleado) {
        if (nodo == null) {
            return new NodoEmpleado(empleado);
        }

        // Ordenar por departamento para la búsqueda
        int comparacion = empleado.getDepartamento().compareTo(nodo.empleado.getDepartamento());

        if (comparacion < 0) {

```



Estructuras utilizadas

Listas (ArrayList, LinkedList): Se usan para almacenar colecciones ordenadas de objetos, como las tareas asignadas a un empleado. **ArrayList** es una lista dinámica, mientras que **LinkedList** es eficiente para inserciones y eliminaciones en los extremos.

Colas de Prioridad (PriorityQueue): Permite que las tareas se procesen según su prioridad, no por su orden de llegada. Esto es ideal para un sistema de gestión de tareas.

Pilas (Stack): Esta estructura de datos se importa y se podría usar para una función como deshacer una acción. Este dato sigue el principio "último en entrar, primero en salir" (LIFO). Aunque no se usa directamente en este código.

Mapas (HashMap): Almacena pares de clave-valor. Se utiliza en el **GrafoTareas** para mapear una tarea a un conjunto de otras tareas de las que depende, lo que permite una búsqueda rápida.

Conjuntos (HashSet): Almacena elementos únicos sin un orden específico. Se usa en el **GrafoTareas** para guardar las tareas dependientes de una tarea principal, asegurando que no haya duplicados.

Árboles Binarios de Búsqueda (ArbolBinarioEmpleados): Esta estructura de datos jerárquica almacena los empleados de forma organizada, permitiendo una búsqueda, inserción y eliminación eficiente. En este caso, se ordenan por departamento para facilitar la búsqueda.



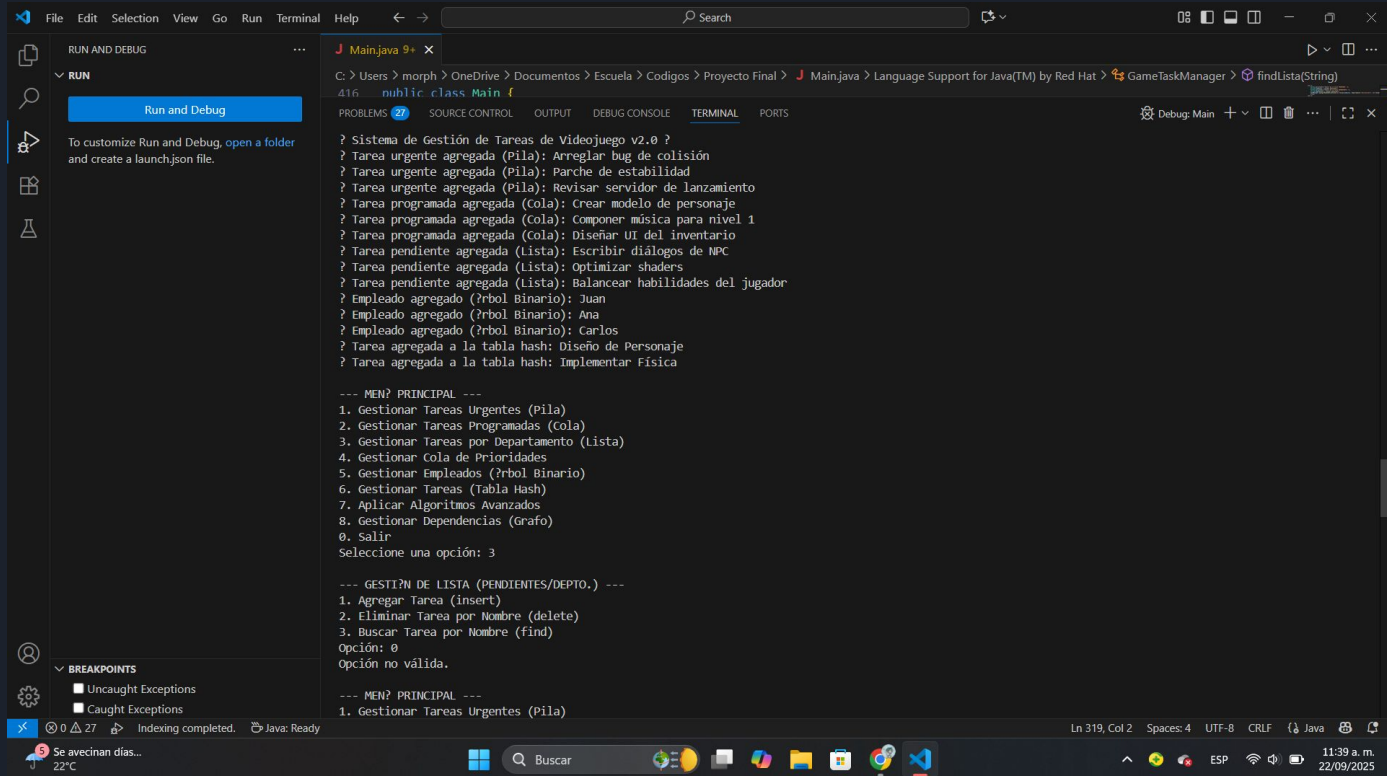
FUNCIÓN GENERAL

En general, el código demuestra cómo se pueden usar las estructuras de datos para modelar un sistema complejo de gestión. Puedes:

1. Crear **tareas** de diferentes tipos (simples o con prioridad).
2. Crear **empleados** y asignarles tareas.
3. Organizar a los empleados en un **árbol binario** para búsquedas rápidas por departamento.
4. Definir las **dependencias** entre tareas usando un **grafo**.

El código es un ejemplo robusto y bien estructurado que muestra la aplicación práctica de conceptos avanzados de programación y estructuras de datos.

FUNCIÓN DE CÓDIGO



File Edit Selection View Go Run Terminal Help

Search

416 public class Main {

RUN AND DEBUG

RUN

Run and Debug

To customize Run and Debug, open a folder and create a launch.json file.

BREAKPOINTS

☐ Uncaught Exceptions

☐ Caught Exceptions

J Main.java 9+ x

C:\> Users > morph > OneDrive > Documentos > Escuela > Codigos > Proyecto Final > J Main.java > Language Support for Java(TM) by Red Hat > GameTaskManager > findLista(String)

PROBLEMS 27 SOURCE CONTROL OUTPUT DEBUG CONSOLE TERMINAL PORTS

Debug: Main

1. Gestionar Tareas Urgentes (Pila)
2. Gestionar Tareas Programadas (cola)
3. Gestionar Tareas por Departamento (Lista)
4. Gestionar Cola de Prioridades
5. Gestionar Empleados (?rbol Binario)
6. Gestionar Tareas (Tabla Hash)
7. Aplicar Algoritmos Avanzados
8. Gestionar Dependencias (Grafo)
0. Salir
Seleccione una opción: 6

--- GESTION DE TAREAS (TABLA HASH) ---
1. Agregar Tarea
2. Buscar Tarea por Nombre
Opción: 0
Opción no válida.

--- MENU PRINCIPAL ---
1. Gestionar Tareas Urgentes (Pila)
2. Gestionar Tareas Programadas (cola)
3. Gestionar Tareas por Departamento (Lista)
4. Gestionar Cola de Prioridades
5. Gestionar Empleados (?rbol Binario)
6. Gestionar Tareas (Tabla Hash)
7. Aplicar Algoritmos Avanzados
8. Gestionar Dependencias (Grafo)
0. Salir
Seleccione una opción: 7

--- ALGORITMOS AVANZADOS ---
1. Calcular tiempo total estimado (Recursividad)
2. Optimizar distribución de tareas ('Divide y Vencerás')
3. Ordenar tareas por prioridad y fecha
4. Búsqueda Binaria de tarea
Opción: 0
Opción no válida.

Ln 319, Col 2 Spaces: 4 UTF-8 CRLF Java

Indice ipc +0.69%

Buscar

11:41 a.m. 22/09/2025

The image shows a screenshot of an IDE (Integrated Development Environment) with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The left sidebar has icons for Explorer, Search, Run and Debug, and Breakpoints. The main editor area shows a Java file named 'Main.java' with the following code:

```
416 public class Main {  
    2. Optimizar distribución de tareas ('Divide y Vencerás')  
    3. Ordenar tareas por prioridad y fecha  
    4. Búsqueda Binaria de tarea  
    Opción: 0  
    Opción no válida.  
  
    --- MENÚ PRINCIPAL ---  
    1. Gestionar Tareas Urgentes (Pila)  
    2. Gestionar Tareas Programadas (cola)  
    3. Gestionar Tareas por Departamento (Lista)  
    4. Gestionar Cola de Prioridades  
    5. Gestionar Empleados (Arbol Binario)  
    6. Gestionar Tareas (Tabla Hash)  
    7. Aplicar Algoritmos Avanzados  
    8. Gestionar Dependencias (Grafo)  
    0. Salir  
    Seleccione una opción: 8  
  
    --- GESTIÓN DE DEPENDENCIAS (GRAFO) ---  
    1. Agregar Dependencia de Tarea  
    2. Ver todas las dependencias  
    Opción: 0  
    Opción no válida.  
  
    --- MENÚ PRINCIPAL ---  
    1. Gestionar Tareas Urgentes (Pila)  
    2. Gestionar Tareas Programadas (cola)  
    3. Gestionar Tareas por Departamento (Lista)  
    4. Gestionar Cola de Prioridades  
    5. Gestionar Empleados (Arbol Binario)  
    6. Gestionar Tareas (Tabla Hash)  
    7. Aplicar Algoritmos Avanzados  
    8. Gestionar Dependencias (Grafo)  
    0. Salir  
    Seleccione una opción: 0  
    ? Saliendo del sistema. ¡Adiós!  
    PS C:\Users\morph>
```

The terminal output shows the program running and displaying a menu of tasks. The status bar at the bottom indicates 'Ln 319, Col 2', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java'. The system tray at the bottom right shows the date and time: '11:40 a. m. 22/09/2025'.



GRACIAS POR
SU ATENCIÓN

Con esto concluyo la presentación de mi
código